

Naukri.com Job Auto-Applier

An Autonomous, AI-Assisted Job Search & Application System

Project Report

Prepared by: Vidur Sharma

Live Dashboard: <https://naukri-bot.duckdns.org>

Repository: github.com/vidur0001/naukri-jobs-auto-applier

1. Overview

The Naukri.com Job Auto-Applier is an autonomous, AI-assisted system that automates the end-to-end job search and application workflow on Naukri.com - from job discovery to AI relevance scoring to auto-applying to notifying the user - while remaining safe, controllable, and human-supervised at every critical step. It runs 24/7 in the cloud and exposes a live public dashboard so the system's activity can be monitored from any device.

2. Problem Statement

Manually searching Naukri.com across multiple roles and cities, reading every job description, judging relevance, and filling repetitive application forms is time-consuming and error-prone. Good roles get missed in the noise, and the same form fields and open-ended questions ('Tell us about yourself', 'How do you handle failure?') are answered over and over. The goal was to remove this repetitive load with automation and AI judgment, without losing human oversight over what actually gets submitted, and without exposing credentials or control of the bot to anyone else.

3. What Was Built

- **Automated job search & application engine (bot.py):** Playwright-driven browser automation that loops through configured roles x cities, de-duplicates already-seen jobs, and extracts full job descriptions.
- **AI relevance scoring & summarization (ai_helper.py):** Uses the Groq LLM to score every job 0-100 against a structured resume profile; low-relevance jobs are silently skipped so only meaningful jobs reach the user.
- **Safety-windowed auto-apply:** A 'Skip' button in the notification email gives a grace period before the bot applies automatically; a global Pause/Resume switch halts all applying independent of any single job's decision.
- **AI-powered questionnaire answering:** Open-ended application questions are answered in natural, first-person language by the LLM, grounded strictly in the real resume profile - never fabricated. Structured fields (CTC, notice period, skills) are filled from configuration.
- **Bot-detection resilience:** Runs headed Chromium via Xvfb with stealth patches to avoid Naukri's Akamai WAF. On a hard CAPTCHA/verification challenge, an email link opens a live remote browser view (noVNC) so a human can solve it in seconds.
- **Remote Naukri login capture:** A 'Login to Naukri' flow, streamed over the same noVNC channel, lets the session be created or refreshed from any device. No password is ever sent to or stored by the server.
- **Live public dashboard (dashboard/):** A React + Vite single-page app showing real-time bot status, daily statistics, and application history, polling a JSON API every few seconds.
- **Secured Flask backend (app.py):** Every control action (start, stop, pause, resume, decide, edit, login) requires a per-deployment secret token, automatically embedded in email links. Read-only dashboard and API routes remain public with no token required.

- **Production hosting:** Dockerized and deployed on an AWS EC2 instance with a static Elastic IP; a Caddy reverse-proxy container provides free, auto-renewing HTTPS on a DuckDNS domain so the dashboard has a stable, shareable public URL.

4. Why These Design Choices

- **Human-in-the-loop over full autonomy:** Applying to jobs has real consequences (irrelevant applications hurt visibility with recruiters), so every apply action is skippable and pausable rather than silently automatic.
- **Remote browser login (noVNC) over storing credentials:** Avoids the server ever handling the user's Naukri password - the trust model is identical to visiting naukri.com directly in a normal browser.
- **Token-gated webhooks over open routes:** Once the dashboard became public via a custom domain, every control action needed explicit authorization, independent of the obscurity of the URL.
- **Static dashboard + polling over WebSockets:** Simpler to build and host reliably on a single EC2 instance; bot/job data changes at a human-observable pace (seconds), so periodic polling is sufficient and avoids extra infrastructure.
- **LLM-based scoring over keyword filters:** Job descriptions vary widely in phrasing; an LLM comparing the JD against a structured resume profile catches genuine relevance that simple keyword matching would miss.

5. System Architecture

The system is composed of four cooperating layers, all running inside a single Docker Compose stack on one EC2 instance:

Layer	Component(s)	Responsibility
Automation Engine	bot.py	Playwright browser automation: search, scrape, score, apply, form-fill, email
AI Layer	ai_helper.py + Groq LLM	Relevance scoring, JD summarization, natural-language questionnaire answers
Backend / Control Plane	app.py (Flask)	Start/stop/pause control, decision & clarification webhooks, JSON API, dashboard hosting, token security
Frontend	dashboard/ (React + Vite)	Live status, stats, application history, session/login panel
Infrastructure	Docker, Caddy, DuckDNS, AWS EC2	Containerization, automatic HTTPS, stable public domain, always-on hosting

Request & Data Flow

1. bot.py iterates every (role, location) pair from config.json and searches Naukri.
2. New job descriptions are sent to the Groq LLM for a 0-100 relevance score against profile.json.
3. Jobs passing the threshold trigger an email with job details and a Skip button (grace period).
4. If not skipped (and auto-apply is not globally paused), the bot applies: filling structured fields from config.json and routing open-ended questions through the LLM.
5. Every event (found, applied, skipped) is logged to applications_log.json and daily_stats.json, which the Flask JSON API (/api/status, /api/stats, /api/applications) reads and the React dashboard polls and renders.
6. All control/action requests (from email links or the dashboard) pass through _require_challenge_token() in app.py, which checks a secret token before executing anything.

6. Security Measures

- **Token-gated control routes:** /start, /stop, /auto-apply/pause, /auto-apply/resume, /decide/<id>, /clarify/<id>, /edit, /solve-challenge, and /login/* all require a matching CHALLENGE_ACCESS_TOKEN via query string or hidden form field; requests without it receive 403 Forbidden.
- **No credential exposure:** Naukri login happens in a real browser over noVNC - the password is typed directly into Naukri's page, never transmitted to or stored by the Flask backend.
- **Read-only public surface:** /dashboard and /api/* endpoints expose no secrets and no control actions, so the dashboard can be safely public while the bot itself stays protected.
- **Automatic HTTPS:** Caddy provisions and renews a Let's Encrypt certificate automatically for the public domain, so all traffic (including tokens in email links) is encrypted in transit.
- **Secrets kept out of git:** .env, config.json, naukri_storage_state.json, SSH keys, and session/profile directories are all git-ignored and supplied per-environment.

7. Deployment

Hosted on an AWS EC2 (Ubuntu) instance in eu-north-1. Docker Compose runs two services: naukri-bot (the Flask app + Playwright automation, restart: unless-stopped) and caddy (reverse proxy + automatic HTTPS). An Elastic IP keeps the public IP - and therefore every email webhook link - constant across instance stops, restarts, and reboots. A free DuckDNS subdomain maps to the Elastic IP, and Caddy issues a Let's Encrypt certificate for it, giving a stable <https://naukri-bot.duckdns.org> URL that redirects to the dashboard.

8. Outcome

A fully autonomous, always-on job-application pipeline combining browser automation, LLM-based judgment, and human override, deployed securely with a live public dashboard that demonstrates the system in action end-to-end via a single shareable HTTPS link.

9. Tech Stack

Category	Technologies
Automation	Python, Playwright (async API), Xvfb (headed browser in Docker)
AI / LLM	Groq API (relevance scoring, summarization, questionnaire answers)
Backend	Flask, python-dotenv, Jinja2 (email templates)
Frontend	React, Vite, custom polling hooks
Infrastructure	Docker, Docker Compose, AWS EC2, Elastic IP
Networking / Security	Caddy (reverse proxy + auto HTTPS), DuckDNS, token-based auth, noVNC/x11vnc